EventFlow Backend API Documentation

This document describes the API implemented in this repository so frontend developers can integrate without guessing behavior.

Important

- All endpoint details below are taken from current route and controller code.
- Some response keys have typos or inconsistent casing in the current implementation. Those are documented exactly as returned.

Base URLs

- Root health: / and /api/v1
- API base: /api/v1

Local URL example

- http://localhost:4000/api/v1

Run Locally

1. Install dependencies

```
npm install
```

2. Create a .env file and define the environment variables listed in this document.
3. Start the server

```
npm run dev
```

4. Verify API is up

```
GET http://localhost:4000/api/v1
```

Authentication

- Protected endpoints require Authorization header in this format:

```
Bearer <token>
```

- Tokens are issued by signup/login.
- Logout blacklists a token by saving it to the blacklist collection.

Content Types

- Most endpoints: application/json
- Event creation with image upload: multipart/form-data (file field name must be image)

Environment Variables Used In Code

- PORT
- NODE_ENV
- jwtSecret
- jwtExpiresIn
- SECRET_KEY
- cloudinary_name
- cloudinary_api_key
- cloudinary_api_secret
- EMAIL_HOST
- EMAIL_PORT
- EMAIL_USERNAME
- EMAIL_PASSWORD

Note about database bootstrap

- No database connection initialization file is present in the current workspace files.

Common Response Pattern

- Success responses are typically JSON with status/message and data.
- Production error middleware often returns:

```
{
      "status": "faIL",
      "message": "..."
}
```

- Some controllers return custom error shapes directly.

```
------------------------------------------------
1) Auth Endpoints
------------------------------------------------

1. POST /auth/signup
- Full path: /api/v1/auth/signup
- Auth: Public
- Body (JSON):
      {
            "email": "user@example.com",
            "username": "john",
            "password": "secret123"
      }
- Validation:
      - email must be valid
      - username required
      - password min length 6
- Success (201):
      {
            "status": "success",
            "messgae": "User created successfully",
            "user": { ... },
            "token": "..."
      }

2. POST /auth/login
- Full path: /api/v1/auth/login
- Auth: Public
- Body (JSON):
      {
            "email": "user@example.com",
            "password": "secret123"
      }
- Success (200):
      {
            "status": "success",
            "message": "Login successful",
            "user": { ... },
            "token": "..."
      }

3. POST /auth/logout
- Full path: /api/v1/auth/logout
- Auth: Public (token is sent in body)
- Body (JSON):
      {
            "token": "jwt-token-to-blacklist"
      }
- Success (200):
      {
            "status": "success",
            "message": "Logout successful"
      }

------------------------------------------------
2) User Endpoints
------------------------------------------------

1. GET /user/profile
- Full path: /api/v1/user/profile
- Auth: Required
- Success (200):
      {
```

```
            "status": "success",
            "message": "User profile was successfully fetched!",
            "userProfile": { ... }
      }

2. PATCH /user/profile
- Full path: /api/v1/user/profile
- Auth: Required
- Body (JSON):
      {
            "oldUsername": "old-name",
            "newUsername": "new-name"
      }
- Success (200):
      {
            "status": "success",
            "message": "User profile updated successfully",
            "user": { ... }
      }

3. DELETE /user/profile
- Full path: /api/v1/user/profile
- Auth: Required
- Success (200):
      {
            "status": "success",
            "message": "Profile deleted successfully"
      }

4. PATCH /user/changepassword
- Full path: /api/v1/user/changepassword
- Auth: Required
- Body (JSON):
      {
            "oldPassword": "old-secret",
            "newPassword": "new-secret"
      }
- Success (200):
      {
            "status": "success",
            "message": "Password changed successfully"
      }

5. GET /user/my-events
- Full path: /api/v1/user/my-events
- Auth: Required
- Success (200):
      {
            "status": "success",
            "message": "User's events fetched successfully",
            "result": 1,
            "events": [ ... ]
      }

6. GET /user/my-transactions
- Full path: /api/v1/user/my-transactions
- Auth: Required
- Success (200):
      {
            "transactions": [
                  {
                        "eventName": "...",
                        "amount": 1000,
                        "datePaid": "YYYY-MM-DD",
```

```
                        "status": "pending|completed|cancelled"
                  }
            ]
      }
```

7. GET /user/Userregisteringformyevent/:eventId
- Full path: /api/v1/user/Userregisteringformyevent/:eventId
- Auth: Required
- Description: Event owner fetches registrations for their event.
- Success (200):

```
      {
            "userRegistrations": [
                  {
                        "userName": "...",
                        "ticketType": "Regular|VIP",
                        "status": "...",
                        "ticketPrice": 100,
                        "numberOfSeats": 2,
                        "pricePaid": 200
                  }
            ]
      }
```

8. GET /user/myregister-event
- Full path: /api/v1/user/myregister-event
- Auth: Required
- Success (200):

```
      {
            "message": "My registered by the user",
            "events": [
                  {
                        "eventName": "...",
                        "eventDate": "YYYY-MM-DD",
                        "eventLocation": "...",
                        "eventVenue": "...",
                        "eventCategory": "...",
                        "daysUntilEvent": 10,
                        "registrationStatus": "...",
                        "numberOfSeats": 1,
                        "ticketPrice": 0,
                        "ticketType": "Regular"
                  }
            ]
      }
```

9. POST /user/registerforevent
- Full path: /api/v1/user/registerforevent
- Auth: Required
- Body (JSON):

```
      {
            "userId": "<user-id>",
            "ticketId": "<ticket-id>",
            "eventId": "<event-id>",
            "email": "payer-or-user-email@example.com",
            "numberOfSeats": 2,
            "ticketType": "Regular"
      }
```

- Behavior:
      - If ticket price > 0: initializes Paystack payment and returns paymentUrl.
      - If ticket price is 0: marks as completed and deducts seats immediately.
- Success (200):

```
      {
            "message": "Registration successful",
```

```
            "registration": { ... },
            "paymentUrl": "https://... or null"
      }

-------------------------------------------------
3) Event Endpoints
-------------------------------------------------

1. POST /event/createEvent
- Full path: /api/v1/event/createEvent
- Auth: Required
- Content type: multipart/form-data
- Fields:
      - name (required)
      - description
      - date (required, must be future date)
      - location (required)
      - category (required): sport | travel | conference | business | festival |
music
      - venue (required)
      - image (optional file field)
- Success (201):
      {
            "status": "success",
            "message": "Event created successfully",
            "event": { ... }
      }

2. GET /event/getEvent
- Full path: /api/v1/event/getEvent
- Auth: Public
- Success (200):
      {
            "status": "success",
            "results": 10,
            "events": [ ... ]
      }

3. GET /event/:eventId
- Full path: /api/v1/event/:eventId
- Auth: Public
- Success (200):
      {
            "status": "success",
            "event": { ... }
      }

4. PUT /event/:eventId
- Full path: /api/v1/event/:eventId
- Auth: Required (must be event creator)
- Body: partial Event fields to update
- Success (200):
      {
            "status": "success",
            "message": "Event updated successfully",
            "event": { ... }
      }

5. DELETE /event/:eventId
- Full path: /api/v1/event/:eventId
- Auth: Required (must be event creator)
- Success (200):
      {
            "status": "Success",
```

```
            "message": "Event successfully deleted"
      }
```

6. GET /event/events/upcoming
- Full path: /api/v1/event/events/upcoming
- Auth: Public
- Description: Returns events with date greater than last day of current month.
- Success (200):

```
      {
            "status": "success",
            "message": "Upcoming events fetched successfully",
            "result": 2,
            "events": [ ... ]
      }
```

7. GET /event/events/featured
- Full path: /api/v1/event/events/featured
- Auth: Public
- Description: Returns events from now through next 7 days.

8. GET /event/event/usereventvenues
- Full path: /api/v1/event/event/usereventvenues
- Auth: Public
- Success (200):

```
      {
            "userCount": 100,
            "eventCount": 50,
            "venueCount": 12
      }
```

9. GET /event/category/:category
- Full path: /api/v1/event/category/:category
- Auth: Public
- Success (200):

```
      {
            "status": "success",
            "message": "Events fetched successfully",
            "events": [ ... ]
      }
```

10. GET /event/categories/length
- Full path: /api/v1/event/categories/length
- Auth: Public
- Success (200):

```
      {
            "status": "success",
            "message": "Length of categories fetched successfully",
            "categories": [
                  { "category": "music", "length": 4 }
            ]
      }
```

11. GET /event/search/:name/:category/:time
- Full path: /api/v1/event/search/:name/:category/:time
- Auth: Public
- Parameters:
      - name: string or any
      - category: one of categories or any
      - time: today | tomorrow | this_week | next_week | next_month | future
- Success (200):

```
      {
            "status": "success",
            "count": 3,
            "events": [ ... ]
```

```
      }

12. POST /event/createTicket
- Full path: /api/v1/event/createTicket
- Auth: Required
- Body (JSON):
      {
            "eventId": "<event-id>",
            "userId": "<owner-id>",
            "type": "Regular or VIP",
            "sit": 50,
            "price": 0,
            "bankName": "Required when price > 0",
            "bankAccount": 1234567890,
            "bankHolderName": "Required when price > 0"
      }
- Success (201):
      {
            "status": "success",
            "message": "Regular ticket created successfully",
            "ticket": { ... }
      }

13. GET /event/getTicket/:ticketId
- Full path: /api/v1/event/getTicket/:ticketId
- Auth: Public
- Note: Controller treats :ticketId as eventId and fetches all tickets by
eventId.
- Success (200):
      {
            "status": "success",
            "message": "Tickets for the event fetched successfully",
            "resulit": 2,
            "tickets": [ ... ]
      }

14. PUT /event/updateTicket/:ticketId
- Full path: /api/v1/event/updateTicket/:ticketId
- Auth: Required (must own ticket)
- Body:
      - sit, price
      - if existing ticket is paid (price > 0),
bankName/bankAccount/bankHolderName are required
- Success (200):
      {
            "status": "success",
            "message": "Ticket updated successfully",
            "ticket": { ... }
      }

15. DELETE /event/deleteTicket/:ticketId
- Full path: /api/v1/event/deleteTicket/:ticketId
- Auth: Required (must own ticket)
- Success (200):
      {
            "status": "success",
            "message": "Ticket deleted successfully"
      }

------------------------------------------------
4) Payment Webhook Endpoint
------------------------------------------------

Path
```

- /api/v1/payment/webhooks

Notes
- This route is mounted with app.use, so it is not restricted to one HTTP method in routing.
- Intended for Paystack webhook callbacks, not frontend clients.
- Signature validation compares request authorization header to a calculated hash.

Possible Success (200)
```
{
      "status": "success",
      "message": "Payment status updated successfully: success|failed|pending"
}
```

Possible Failure (404)
```
{
      "status": "fail",
      "message": "invalid webhook signature"
}
```

---
5) Frontend Integration Guide
---

1. Authenticate user
- Call signup or login and store token securely in memory/local storage strategy used by your app.

2. Send token on protected requests
- Header:
      Authorization: Bearer <token>

3. Register paid event flow
- Call POST /api/v1/user/registerforevent.
- If response has paymentUrl, redirect user to that URL to complete payment.
- Poll/refetch registered events or transactions after payment completion.

4. Register free event flow
- Same endpoint returns paymentUrl as null and registration completes immediately.

---
6) Quick Endpoint Summary

Auth
- POST /api/v1/auth/signup
- POST /api/v1/auth/login
- POST /api/v1/auth/logout

User
- GET /api/v1/user/profile
- PATCH /api/v1/user/profile
- DELETE /api/v1/user/profile
- PATCH /api/v1/user/changepassword
- GET /api/v1/user/my-events
- GET /api/v1/user/my-transactions
- GET /api/v1/user/Userregisteringformyevent/:eventId
- GET /api/v1/user/myregister-event
- POST /api/v1/user/registerforevent

Event
- POST /api/v1/event/createEvent

- GET /api/v1/event/getEvent
- GET /api/v1/event/:eventId
- PUT /api/v1/event/:eventId
- DELETE /api/v1/event/:eventId
- GET /api/v1/event/events/upcoming
- GET /api/v1/event/events/featured
- GET /api/v1/event/event/usereventvenues
- GET /api/v1/event/category/:category
- GET /api/v1/event/categories/length
- GET /api/v1/event/search/:name/:category/:time
- POST /api/v1/event/createTicket
- GET /api/v1/event/getTicket/:ticketId
- PUT /api/v1/event/updateTicket/:ticketId
- DELETE /api/v1/event/deleteTicket/:ticketId

Webhook
- /api/v1/payment/webhooks